

Technical Architecture and Kinematic Modeling of sOFIa, an Interactive Robotic Artistic System

Juan Mauricio Serna Gómez, Alejandro Rodríguez Veloquio, Francisco Uriel Padilla Pedroza, Diego Rosales Rojas, Jorge A. Lizarraga, Graciela Baca, Luis Enrique González-Jiménez and Luis F. Luque-Vega

Abstract—sOFIa is an interactive robotic artistic system able to establish visual communication between people and a mechatronic array (that is also modular). The platform combines a camera and a PC-based perception interface with an array of actuated hexagonal units. This work documents the Discover and Learn stages of the project by characterizing the reference system, formalizing its computational requirements, defining a preliminary perception-to-actuation architecture, and establishing the kinematic model used by the actuated modules. The implemented vision baseline uses MediaPipe hand landmarks with OpenCV and NumPy to recognize five hand-based interaction gestures, including three dynamic gestures, while optional body-pose association supports multi-user operation. Temporal landmark smoothing, output-rate limiting, a 16-module mapping stage, and a communication interface to the master ESP32 are incorporated as part of the preliminary software architecture. For the mechatronic side, a two-degree-of-freedom rotational model is expressed through sequential rotations and an inverse-kinematics mapping from a target direction to the joint variables. A MATLAB microtest implements a 16-unit honeycomb array following a moving three-dimensional point under explicit joint limits and singularity handling. The resulting methodology provides a traceable baseline for subsequent implementation, integration, and experimental validation of the complete gesture-reaction loop.

Index Terms—human-robot interaction, hand gesture recognition, human pose estimation, inverse kinematics, modular robotics, computer vision, robotic artistic system

I. INTRODUCTION

SOFIA is a robotic artistic interactive system based on intelligent vision. The reference system combines a mechatronic structure formed by actuated hexagonal panels, a PC, and a camera. Its intended interaction is visual: people inside the camera FOV produce body postures, the computational subsystem interprets those postures, and the mechatronic subsystem reacts through coordinated motion.

The technical challenge is not limited to pose estimation or to actuator control alone. The complete interaction loop must detect five postures in one or more persons, map those detections to mechanical reactions, exchange commands wirelessly between the PC and the master ESP32, provide an operator interface for visualization and debugging, and support performance evaluation of the integrated system [1]. These requirements couple computer vision, communication, embedded control, and the kinematics of the actuated surface.

A useful formulation of the project question is: *how can a camera-based multi-person interaction pipeline be integrated with a modular two-degree-of-freedom kinetic array so that hand-based postures and motions can be translated*

into bounded and reproducible mechanical reactions? At the current Discover–Learn stage, the objective is to characterize the reference system, identify the technical decisions that constrain the solution, define a preliminary architecture and risk model, and establish a reproducible kinematic microtest before full hardware/software integration.

The main contribution of this “Corte 1” manuscript is consequently a traceable technical baseline rather than a final validated prototype. It connects the project requirements with a perception-framework selection rationale, a preliminary bill of materials (BOM), the mathematical orientation model of one sOFIa unit, and a simulation protocol for the 16-unit array. These elements define what must be implemented and measured during the subsequent Apply and Build & Validate stages.

A. Related Work

Real-time human-pose estimation provides the perception layer required by sOFIa. OpenPose introduced a bottom-up multi-person approach based on Part Affinity Fields, associating detected body parts with individual people while maintaining real-time operation independent of the number of people in the scene [2]. This directly addresses the need to extract body keypoints from multiple users, although it does not by itself define the application-specific posture vocabulary or the downstream mechatronic response.

MediaPipe was proposed as a framework for constructing perception pipelines with explicit attention to cross-platform deployment, resource consumption, and iterative prototyping [3]. The current sOFIa vision prototype adopts the MediaPipe ecosystem for hand-landmark extraction, while an optional body-pose stage is used to associate detected hands with people in multi-user scenes. BlazePose remains relevant as a lightweight real-time body-pose reference within the same ecosystem [4].

The project definition also identifies YOLO-based pose estimation as a candidate and includes the Ultralytics package in the general software framework [1]. YOLO11 provides pre-trained pose/keypoint variants within the Ultralytics workflow [5]; in the present Corte 1 implementation it is treated as a comparison alternative rather than as the selected gesture-recognition backend. Any future substitution of the vision backend must be justified through measurements under the project’s camera, illumination, and multi-user conditions.

These approaches primarily solve the perception problem: they estimate people, hands, or skeletal keypoints from images or video. sOFIa additionally requires temporal gesture rules,

smoothing against false positives, gesture-to-reaction semantics, a wireless command/acknowledgment layer, and bounded mechanical motion of the hexagonal array. The project gap is therefore an integration gap: adapting established landmark-estimation tools into a reproducible perception-to-actuation pipeline tailored to the sOFia demonstrator.

II. MATERIALS AND METHODS

A. Reference System and Functional Decomposition

The reference architecture contains a PC/camera perception side and a distributed mechatronic side. According to the project specification, the PC acquires video, detects postures, presents the detection state through a GUI, and sends the corresponding reaction command wirelessly to a master ESP32. The mechatronic architecture distributes actuator commands through module electronics that include PCA9685 PWM drivers and an internal RS-485/Modbus-oriented bus between controller boards. The functional chain used in this work is

$$\begin{aligned} &\text{camera} \rightarrow \text{landmarks} \rightarrow \text{gesture rule} \\ &\rightarrow \text{reaction command} \rightarrow \text{module motion.} \end{aligned} \quad (1)$$

Table I converts the project specification into requirements that can later be verified experimentally. No numerical acceptance thresholds are invented at this stage; quantities such as accuracy, response time, and frame rate are to be measured during validation.

TABLE I
PRELIMINARY FUNCTIONAL AND VALIDATION REQUIREMENTS DERIVED FROM THE SOFIA TECHNICAL SPECIFICATION.

ID	Requirement	Verification evidence planned
FR1	Detect five designed hand-based interaction gestures for people inside the camera field of view, while retaining a path for multi-user association.	Live annotated video, per-gesture hit rate, and confusion/error records.
FR2	Include at least three dynamic gestures whose recognition depends on more than one captured image.	Temporal rule traces and test sequences.
FR3	Reduce spurious gesture detections caused by landmark noise and abrupt output changes.	Documented smoothing/debounce rule and false-positive comparison.
FR4	Exchange gesture/reaction commands wirelessly between the PC and the master ESP32.	Message format, acknowledgments, error codes, and communication test.
FR5	Provide a GUI for live video, detection state, execution control, and integration debugging.	Functional interface demonstration without blocking the processing thread.
FR6	Characterize integrated detection performance.	Hit rate, response time, average FPS, and tests with different users and illumination conditions.

B. Vision Framework Selection

The project specification identifies YOLO, MediaPipe, OpenPose, and OpenCV DNN with a TensorFlow model as candidate vision approaches. For the current “Corte 1” prototype, the team selected a Python pipeline based on MediaPipe hand tracking, OpenCV, and NumPy. The selection is motivated by the ability to obtain 21 hand landmarks in real time without training a project-specific neural network, while OpenCV provides camera acquisition and visualization and NumPy supports the coordinate calculations used by the gesture logic. Table II retains the broader alternatives as part

of the technical selection record rather than presenting them as implemented backends.

TABLE II
QUALITATIVE COMPARISON OF VISION ALTERNATIVES CONSIDERED FOR SOFIA AND STATUS IN THE CORTE 1 PROTOTYPE.

Alternative	Relevant capability	Project consideration	Corte 1 status
MediaPipe + OpenCV	Landmark-oriented perception pipeline suitable for real-time interaction [3].	The team’s implementation uses hand landmarks for the gesture vocabulary and can optionally associate hands with detected body poses.	Selected and implemented baseline.
OpenPose	Real-time multi-person 2-D pose with Part Affinity Fields [2].	Strong direct fit to multi-person body-keypoint extraction; integration and computational cost would need evaluation on the target PC.	Literature / comparison alternative.
YOLO11-pose	Unified Ultralytics pose/keypoint workflow [5].	Present in the general project framework, but not used as the current hand-gesture backend.	Comparison alternative.
OpenCV DNN + TensorFlow	Model-dependent inference path within the OpenCV ecosystem.	Flexible, but behavior and resource use depend on the selected network.	Fallback / comparison alternative.

The prototype is implemented in Python with MediaPipe, OpenCV, and NumPy. Exact package and interpreter versions will be pinned in the project repository to preserve reproducibility.

C. Gesture Vocabulary and Mechanical Reactions

The five interactions defined for the vision module are hand-based gestures. Three are dynamic, and two are static configurations with a one-second temporal hold condition. Table III records the gesture semantics and the intended reaction of the kinetic array. These mappings are design specifications for the interaction loop; their recognition accuracy and mechanical execution will be quantified during later validation.

TABLE III
FIVE VISION GESTURES AND INTENDED SOFIA REACTIONS DEFINED FOR THE INTERACTION PROTOTYPE.

Gesture	Detection concept	Intended mechanical reaction
Open-hand vertical displacement	Track an open hand moving upward or downward across successive frames.	Follow the user’s hand in the vertical direction.
Open-hand horizontal displacement	Track an open hand moving left or right across successive frames.	Move from left to right, or in the corresponding horizontal direction, until the action stops.
Pulse: open–closed–open	Detect the temporal sequence of opening, closing, and re-opening the hand.	Produce a pulse-like motion intended to simulate a heartbeat.
Sustained open palm	Keep the open-palm state stable for approximately 1 s.	Move all panels vertically and then horizontally once.
Sustained closed fist	Keep the closed-fist state stable for approximately 1 s.	Open the outer panels while closing the inner panels for approximately 3 s.

D. Real-Time Vision and Interaction Pipeline

The implemented software is organized as a camera-to-actuation pipeline,

$$\begin{aligned} \text{camera} &\rightarrow \text{MediaPipe landmarks} \rightarrow \text{smoothing} \\ &\rightarrow \text{gesture engine} \rightarrow \text{16-module mapping} \quad (2) \\ &\rightarrow \text{master ESP32 interface.} \end{aligned}$$

The camera runs in a dedicated capture thread whose policy is to retain the most recent frame and discard older buffered frames. This favors low interaction latency over exhaustive frame processing. MediaPipe inference is handled asynchronously by the tracker backend, while the main loop draws the overlays, evaluates gesture events, updates the module command matrix, and performs communication. The current front end is an OpenCV visualization/control window, being the development interface rather than the final GUI required in the later integration stage.

Hand detections are smoothed independently using a persistent landmark filter keyed to each observed hand. When optional body-pose tracking is enabled, detected hands are associated with body poses so that the software can maintain a person-related identity in multi-user scenes. This association supports the project requirement for several people without changing the hand-based definition of the five gestures.

A second stabilization stage acts on the 16-module command vector through a slew-rate limiter with a deadband. The purpose of this stage is different from landmark smoothing: the former limits abrupt actuator-command changes, while the latter attenuates measurement jitter before gesture classification. Gesture events can be sent individually to the master ESP32, and the full 16-element command matrix can be transmitted periodically at a configurable rate. The communication object also provides an initial handshake, acknowledgment checking for gesture messages, connection maintenance, and a safe stop operation when the application closes.

E. Vision Microtest and Data-Acquisition Plan

The current application already exposes the variables required for the first vision microtests. When metric logging is enabled, each processed iteration can record timestamp t_k , mean frame rate FPS_k , number of detected hands $N_{h,k}$, emitted gesture g_k , gesture confidence c_k , and loop latency τ_k . These fields define the initial acquisition format:

$$\mathcal{D}_k = \{t_k, FPS_k, N_{h,k}, g_k, c_k, \tau_k\}, \quad (3)$$

For "Corte 1", Eq. (3) defines how performance evidence will be collected; no accuracy, FPS, or latency result is claimed until controlled trials have been executed.

The planned validation will exercise each of the five gestures with different users and illumination conditions, including scenes with more than one person when pose association is enabled. At minimum, the later analysis will report gesture hit rate, false activations, response time, and average FPS, matching the performance categories defined in the project specification [1]. This separation between an implemented acquisition mechanism and future measured results keeps the present section methodological rather than prematurely reporting unverified performance.

F. Preliminary Bill of Materials and Key Components

The demonstrator BOM supplied as a starting point contains two PCA9685 PWM drivers, one ESP32-C3 Super Mini board, 32 MG90S servomotors, one RS-485/Modbus driver, a pair of JST-VH connectors, and the PCB/enclosure assembly.

TABLE IV
PRELIMINARY BILL OF MATERIALS FOR THE SOFIA DEMONSTRATOR

Component	Technical Description	Qty.	Manufacturer / Origin
PCA9685	16-channel, 12-bit PWM controller for servo actuation	2	NXP
ESP32-C3 Super Mini	Wi-Fi/BLE microcontroller used as master controller	1	Espressif
MG90S	Micro servo used for the two rotational DOF of each module	32	YSIDO, China
RS-485 driver	Differential communication interface for the distributed bus	1	Stlxy, China
JST-VH	Power/interconnection connectors, male and female	2	China
PCB + enclosure	Custom electronic integration and mechanical protection	1	Local fabrication

G. Kinematic Model of One sOFia Unit

Each hexagonal unit is modeled as a two-degree-of-freedom rotational mechanism actuated by two servomotors. Let q_1 denote the first rotation about the local x axis and q_2 the second rotation about the displaced local y axis. The orientation of the output platform is [9]

$$\mathbf{R}_{1,2} = \mathbf{R}_x(q_1)\mathbf{R}_y(q_2). \quad (4)$$

The order in Eq. (4) is physically significant because the second servo rotates about an axis already displaced by the first rotation [9].

For an output direction initially aligned with the local $+z$ axis, the corresponding unit vector after the two rotations is

$$\hat{\mathbf{r}}_{2,3} = \begin{bmatrix} \sin q_2 \\ -\cos q_2 \sin q_1 \\ \cos q_1 \cos q_2 \end{bmatrix}. \quad (5)$$

Let $\mathbf{p}$ be the target point and $\mathbf{r}_{0,2}^{(i)}$ the origin of the output frame of module i . The desired pointing direction is

$$\hat{\mathbf{r}}_{2,p}^{(i)} = \frac{\mathbf{p} - \mathbf{r}_{0,2}^{(i)}}{\|\mathbf{p} - \mathbf{r}_{0,2}^{(i)}\|} = [u_x \ u_y \ u_z]^T. \quad (6)$$

Equating Eqs. (5) and (6) gives the inverse-kinematics solution used in the project material [9]:

$$q_2 = \arcsin(u_x), \quad (7)$$

$$q_1 = \text{atan2}(-u_y, u_z). \quad (8)$$

A singular configuration occurs when $u_x = \pm 1$, which implies $\cos q_2 = 0$ and $u_y = u_z = 0$; in that case q_1 is not determined by the pointing direction [9]. For the simulation microtest, the adopted numerical rule is to retain the previous valid q_1 value (or $q_1 = 0$ at initialization) whenever $\sqrt{u_y^2 + u_z^2} < 10^{-9}$. After inverse kinematics, both joint variables are limited to the selected physical range:

$$q_j^{\text{cmd}} = \text{clip}(q_j, -45^\circ, 45^\circ), \quad j \in \{1, 2\}. \quad (9)$$

H. MATLAB Kinematic Microtest

A MATLAB microtest was prepared to exercise the inverse-kinematics mapping before hardware integration. The code defines 16 module origins in a centered honeycomb arrangement with a pitch of 0.12 m. The fixed dimensions are $z_{1,2} = 0.020$ m and $z_{2,3} = 0.030$ m, and both joint variables are constrained to $\pm 45^\circ$. The moving target follows

$$\mathbf{p}(t) = \begin{bmatrix} 0.15 \cos(0.3t) \\ 0.15 \sin(0.3t) \\ 0.40 \end{bmatrix} \text{ m}, \quad (10)$$

for 25 s with a simulation step of 0.4 s. These values reproduce the uploaded simulation configuration.

Algorithm 1 summarizes the language-independent computation applied to each module. The corrected MATLAB source accompanying this draft uses the same rotation order as Eq. (4) for visualization.

Algorithm 1 Inverse-kinematics update for one sOFIa unit

Require: Module origin $\mathbf{r}_{0,2}$, target $\mathbf{p}$, joint limits

$q_{j,\min}, q_{j,\max}$

Ensure: Bounded commands $q_1^{\text{cmd}}, q_2^{\text{cmd}}$

- 1: $\mathbf{d} \leftarrow \mathbf{p} - \mathbf{r}_{0,2}$
 - 2: $\mathbf{u} \leftarrow \mathbf{d} / \|\mathbf{d}\|$
 - 3: $q_2 \leftarrow \arcsin(\text{clip}(u_x, -1, 1))$
 - 4: **if** $\sqrt{u_y^2 + u_z^2} < 10^{-9}$ **then**
 - 5: $q_1 \leftarrow q_{1,\text{previous}}$ ▷ or zero at initialization
 - 6: **else**
 - 7: $q_1 \leftarrow \text{atan2}(-u_y, u_z)$
 - 8: **end if**
 - 9: $q_1^{\text{cmd}} \leftarrow \text{clip}(q_1, q_{1,\min}, q_{1,\max})$
 - 10: $q_2^{\text{cmd}} \leftarrow \text{clip}(q_2, q_{2,\min}, q_{2,\max})$
 - 11: **return** $q_1^{\text{cmd}}, q_2^{\text{cmd}}$
-

The verification metric must compare the desired direction against the direction actually reconstructed from the bounded joint commands, not against a copy of the original target vector. For module i and frame k , the angular pointing error is therefore defined as

$$e_\theta^{(i,k)} = \cos^{-1} \left(\text{clip} \left[\hat{\mathbf{r}}_{2,3}^{(i,k)T} \hat{\mathbf{r}}_{2,p}^{(i,k)}, -1, 1 \right] \right). \quad (11)$$

This metric will expose any error caused by saturation, singularity handling, numerical precision, or an inconsistent rotation convention.

Figure 1 shows four frames already produced by the current simulation. They are treated here as a microtest visualization, not as final experimental validation of the complete system.

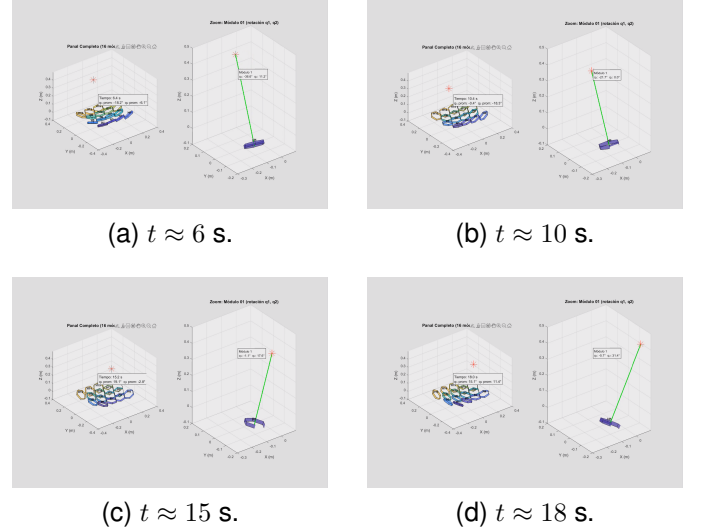


Fig. 1. MATLAB microtest of the 16-unit array following the circular target.

I. Preliminary Integration Risks and Mitigation

Table V records risks already implied by the project requirements and kinematic model. The intent is to make each risk observable and testable during later integration rather than to treat it as an undocumented implementation detail.

TABLE V
PRELIMINARY TECHNICAL RISK REGISTER FOR THE SOFIA INTERACTION LOOP.

Risk	Technical effect	Planned mitigation / evidence
Noisy hand landmarks or rapid classification changes	False gesture triggers and unstable mechanical reactions.	Per-hand landmark smoothing, temporal gesture rules, and output slew limiting; compare false activations before/after stabilization.
Camera unavailable	Perception loop cannot provide a valid command source.	Exception state in the GUI; stop/hold reaction and allow controlled recovery.
ESP32 disconnection or wireless loss	Command delivery is uncertain or stale.	Message acknowledgments, timeout/error codes, and automatic reconnection in the PC/ESP32 protocol.
Joint command outside physical range	Servo saturation and mismatch between desired and achieved pointing direction.	Explicit $\pm 45^\circ$ clipping and angular-error measurement using Eq. (11).
Inverse-kinematics singularity	q_1 becomes undetermined when the target direction aligns with the local x axis.	Detect $u_y \approx u_z \approx 0$ and retain the previous valid q_1 command.
Multi-user ambiguity	A valid hand gesture may be associated with the wrong person or command context.	Associate detected hands with body poses before applying user-specific gesture context; include multi-user scenes in validation.

AUTHOR CONTRIBUTIONS

Individual contribution statements are pending reconciliation with the team's GitHub/bitacora record and must be completed by the authors before submission.

DATA AVAILABILITY

The source code, simulation files, and supporting material developed for the sOFIa project are available at: https://github.com/Mauaza/sOFIa-LabDST_T4/tree/main

ACKNOWLEDGMENT

The authors acknowledge the Laboratorio de Desarrollo de Soluciones Tecnológicas (LabDST) at ITESO for the sOFIA reference platform and technical project material.

The authors also acknowledge the use of generative artificial intelligence tools to assist with the organization and structuring of the manuscript, as well as as a supplementary tool during the research process. All technical decisions, analyses, implementations, verification of information, and final content remain the responsibility of the authors.

CONFLICTS OF INTEREST

Conflict-of-interest status must be confirmed by all authors before final submission.

REFERENCES

- [1] Laboratorio de Desarrollo de Soluciones Tecnológicas (LabDST), “sOFIA: Definición del eje técnico de sistemas computacionales,” ITESO, Project technical specification, 2026.
- [2] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, “Realtime multi-person 2d pose estimation using part affinity fields,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 7291–7299.
- [3] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, “MediaPipe: A framework for building perception pipelines,” *arXiv preprint arXiv:1906.08172*, 2019.
- [4] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “BlazePose: On-device real-time body pose tracking,” *arXiv preprint arXiv:2006.10204*, 2020.
- [5] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Software and documentation, 2024, version 11.0.0; accessed 2026-09-21. [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>
- [6] J. A. Lizarraga, “Como documentar una lista de materiales (BOM),” Laboratorio de Desarrollo de Soluciones Tecnológicas, ITESO, Technical session notes, 2026.
- [7] NXP Semiconductors, *PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller*, NXP Semiconductors, manufacturer datasheet; accessed 2026-09-21. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf>
- [8] Espressif Systems, *ESP32-C3 Series Datasheet*, Espressif Systems, manufacturer datasheet; accessed 2026-09-21. [Online]. Available: https://documentation.espressif.com/esp32-c3_datasheet_en.pdf
- [9] J. A. Lizarraga, “Modelado cinemático de una unidad SOFIA,” Laboratorio de Desarrollo de Soluciones Tecnológicas, ITESO, Technical session notes, 2026.